S2 Text. Agent setup and prompts

This text gives details of how AI agents ran the BURROW project. It complements the Methods section "Agent-based project execution" and Table 4 of the main text. Sources are the repository files manuscript/AGENT_SETUP.md (written by the PI agent from its session record), RESEARCH_TREE.md (research tree and decision log), LESSONS.md, reviews/ and the git history. Git commit times are the authoritative timeline; times are Central European Time (CET).

1. Environment

The agents ran Claude Code (models Claude Opus 5.5 and Claude Sonnet 5.5) inside a Singularity container on one node of the ETH Euler cluster. The node had a 64-core AMD EPYC 9554 processor and four NVIDIA RTX 4090 GPUs, and it was shared with other users' jobs. The prompt stated 128 GB of memory, but the effective container limit was about 79 GB, which the agents discovered when a training run was killed. Read-only mounts provided the AlphaFold database (v6) and a full PDB mirror. There was no root access, no Java, no CUDA compiler and no conda at the start; the agents installed tools with uv and brought their own Java runtime for P2Rank. Permission prompts were switched off; the mounted directories were the security boundary. The agents could commit to git but not push.

2. What the human did

A human (the user) set up the container, the hardware allocation and the data mounts, and wrote the initial prompt (Section 8). After the project's first completion, the human sent four messages (reproduced verbatim in Section 9):

1. three questions, about multimer handling and testing, about GPU use, and about whether all ideas for speed and accuracy had been exhausted;
2. a time extension of 3–4 h;
3. a question whether the work was finished;
4. a request for this manuscript and a release repository, which pointed to the edit-for-clarity writing guide.

The environment also restarted the session twice with an empty "Continue" message. The human did not write code, choose methods, select or edit results, or edit text.

3. The PI agent

The PI agent was one long Claude Opus 5.5 session. Its tasks were to plan, delegate, review, integrate and decide. It did not write method code. It owned DESIGN.md, the pre-registration amendments, RESEARCH_TREE.md, LESSONS.md and all git commits. The research tree used the status legend required by the prompt and carried a dated decision log (D0–D19 for the project, D20 for the manuscript) and a two-paragraph status summary at each milestone (M0–M5). The PI read agents' summaries rather than raw logs, to keep its own context small, and it used the tree to resume after interruptions.

4. Sub-agents

Each sub-agent was a fresh Claude Code agent with its own context. The PI launched it with a written brief that stated the goal, the files and directories it owned, the expected deliverables, and the project rules (no commits, no test-set access outside the gated script, detached jobs with logs for anything longer than

about 5 minutes, verification on toy data with known answers). The PI could send follow-up messages to a running or finished agent to continue its work with its context intact. S1 Table lists all 30 roles (25 during the project and 5 for the release repository and this manuscript); the literature reviewer also used two research sub-agents. Several roles were served by the same long-lived agent that was resumed many times, for example the evaluation engineer, the method developer and the geometry and HPC engineer. Agent briefs and session transcripts were not archived as files, so individual delegation steps cannot be audited from the repository. Opus was used for design, method, data, verification and manuscript roles; Sonnet for the baselines engineer, the leakage auditor and the two process retrospectives. The Claude Code version, tool settings and token usage were not recorded. Two "skills" (packaged instructions) were used in the manuscript phase: edit-for-clarity for the text and dataviz for the figures.

Review rounds. The design went through two review rounds, below the cap of three that the prompt allowed. Round 1 had six independent reviewers (external PI; prior work and data; statistics; algorithms and compute; AI-agent failure modes; biophysics and AlphaFold). Round 2 had two (external PI with statistics; data with agent risks). Each reviewer proposed concrete changes within the project's scope or stated that none were needed, and DESIGN.md §8 maps each review to the changes it caused.

Independence rules. A result was trusted only after an agent that did not produce it had checked it. The leakage auditor re-implemented the leakage check from scratch without the builder's scripts. The dev reviewer re-checked dev numbers and the selection protocol. The test verifier recomputed the headline test numbers without importing project code, ran the LIGYSIS authors' evaluation functions for C1, and checked model checksums against all 96 test run records. The external report reviewer assessed the report against the result files and the prompt. A number-checking script resolves every number in the report against committed files (1,724 numbers passed at the final check).

Gating and ledger. Test sets could be scored only through scripts/run_test_eval.py, which refuses modified tracked files, records the commit and the checksums of the model, scoring code, pre-registration, ground truth and predictions, refuses to overwrite, caps non-final versions at three, and appends to results/TEST_LEDGER.md. Training code could not read test labels. Predictions for frozen versions were generated from isolated git worktrees, so that development edits on the main branch could not change them.

File ownership. Each agent owned a list of paths. Only the PI committed, with explicit paths. Agents never ran `git add`, `git commit` or `git reset`.

5. Timeline and interruptions

- Day 1, 14:54: start. 15:26 literature review and design v1; 15:35 design v2 after six reviews; 15:39 design v3 and frozen pre-registration after two reviews. Amendments A1–A3 at 15:48, 16:46 and 20:39, each before the relevant predictions existed.
- About 16:18 and 16:29: two session restarts. All running agents were resumed from their files; no work was lost.
- 16:34: LIGYSIS reproduction gate passed (33 of 33 variants). 18:10: training and dev sets built. 18:22: the independent leakage audit failed (marginal); 186 entries were removed (D6).
- 20:10: first dev results. 21:29: FREEZE v1. 23:09: FREEZE v2, adopted before any v1 test score was computed (v1 prediction files already existed).
- Day 2, 00:27: test v1 scored. 01:14: test v2 scored. 01:41: the v3 interface cycle was negative.
- About 02:40–07:10: an API usage limit paused all agents. Detached jobs completed during the pause.
- 07:10–10:00: descriptive analyses, report, external review and the fast preset.
- 11:31–12:35: user-granted extension (speed work, isolated-chain dimers, per-region output rule).
- Afterwards: manuscript and release repository.

6. Failure modes of the agents that were caught

- The training-set builder's own leakage check passed, but the independent audit found residual leakage (7 assembly-partner chains missed, 85 borderline hits).
- A GPU kernel received a floating-point threshold in single precision. Outputs differed only for atoms at pLDDT exactly 70. An exactness test caught it, it was fixed, and AlphaFold-mode test predictions were regenerated on the CPU path.
- A broad `pkill -f` killed another agent's test job, and a `ps | grep | kill` pattern killed an agent's own shell four times.
- An agent's staging left 169 MB of unreachable git objects.
- A filter bug made one descriptive table use all dimer sites instead of interface sites. The external report reviewer found it.
- Timestamps written from memory were 20–50 minutes off; git history was used instead.
- The output floor of v2 improved the headline metric mainly by shortening output lists, a property of the LIGYSIS closest-pocket rule. Reviewers flagged this, and it is reported.
- Eight idle Java processes left from early smoke tests (about 16 GB of memory) could not be killed for lack of permission and were reported to the user.

7. Process lessons (final LESSONS.md, 15 rules)

1. Inspect mounts, tools and real resource limits before planning.
2. Only the PI commits, with explicit paths.
3. Commit at each hand-off and at least hourly, one commit per result; take times from git or the clock.
4. Keep one source of truth for constants.
5. Run every job longer than about 5 minutes detached, with a log and a completion marker; cap workers on a shared node.
6. After a restart or pause, check git status and disk and resume agents with their file lists.
7. Kill only process IDs stored at launch.
8. Reproduce published numbers before building on an evaluation harness.
9. Data and leakage checks are done only when an independent re-implementation agrees.
10. Pre-register gates, endpoints and the policy on output-list length; log every deviation as a dated decision.
11. Every mechanism or negative claim needs a control; never tune a metric's leniency.
12. Break headline gains down by composition (ligand class, site type) before claiming them.
13. Select on train cross-validation, touch dev rarely, and disclose any test-informed steering.
14. Time-box exploratory branches with a kill criterion; optimise feature-dependent code only after the freeze.
15. Run frozen-version test predictions from an isolated worktree, and test exactness at every boundary value.

8. The project prompt (verbatim)

The following is the content of PROMPT.md, which the PI agent saved verbatim at the start of the project.

```
/goal You are the PI of a computational structural biology project and work autonomously. You lead a team of
↪  agents (Opus 5.5 or Sonnet 5.5, as you see fit). You delegate, review, integrate and decide; you keep the
↪  research record. The goal is the best possible outcome, not the fastest completion. Make decisions
↪  yourself, document why, and keep going.


The goal of this project is to build a novel, ultra-fast, state-of-the-art protein binding pocket detection
↪  and ranking method. It must clearly beat other existing baseline methods like P2Rank and others. We also
↪  want this method to be applicable for homo- and heterodimer structures. Explore which benchmarks or
↪  stratifications are suitable for this sub-goal.

Explore various approaches for identifying, scoring and ranking pockets using explicit 3D geometry, physics,
↪  and analytical biophysics or other approaches. Experiment with representation and feature creation.

True Generalization: Your method should generalize across diverse protein families, unseen folds, apo-to-holo
↪  transitions, and cryptic or allosteric sites-not just memorize common co-factor shapes. Large deep
↪  learning based methods are particularly prone to overfitting. To this end you will need to develop a
↪  robust benchmarking approach. Please try to use established benchmarks for pocket detection and ranking.
↪  Make sure to perform fair comparisons with existing methods. Some example datasets to look use are
↪  LIGYSIS, COACH420 and HOLO4K, scPDB, CryptoBench, HOTPocket, AF2BIND. This is an important part of the
↪  project so try to develop a meaningful comparison.

Dedicated AlphaFold Mode (`--af-mode`): A major application of this tool is proteome-wide pocket discovery on
↪  AlphaFold2/AlphaFold3 structures. Standard pocket finders can fail on AF structures due to:
   - Low-confidence disordered regions (IDRs / "spaghetti loops" with low pLDDT) packing against the core and
   ↪  creating hallucinated pockets or occluding real ones.
   - Apo-like side-chain conformations partially blocking binding clefts.
   - Uncertain inter-domain orientations (high PAE) generating fake inter-domain cavities.
   You must design and validate a dedicated mode that leverages B-factor column pLDDT values (and optionally
   ↪  PAE matrices or structural packing heuristics) to handle confidence weighting, loop pruning/softening,
   ↪  and flexible side-chain tolerance.

Dual CPU & GPU Optimization: The method must include a clean, fast CPU implementation (e.g., vectorized
↪  NumPy/SciPy, Numba, or C++/Cython/Rust extensions) AND a GPU-accelerated backend (e.g., PyTorch tensor
↪  ops, CuPy, or Triton/CUDA kernels without neural networks) capable of high-throughput screening across
↪  thousands of structures.

Check the local directory first to inspect available datasets and hardware resources (GPU/CPU specs).


We will come back and revisit progress in 24 hours so please aim to work within that time budget.

Briefing: you are running inside a Singularity container on the ETH Euler cluster
The container is the sandbox. Permission prompts are switched off. The mounted directories are the real
↪  boundary: nothing else exists inside, and read-only mounts reject writes at the kernel level. If a path
↪  you expect is missing, it was not mounted - say so.
No root. apt-get cannot install anything. Use uv, as a fallback use conda or pixi
Git: commit, do not push. There is no push credential. Commit in the read-write code mount with the preset
↪  identity (a Co-Authored-By trailer is added automatically), leave the work on a branch, and tell the user
↪  what to push.
Where to write. Large or generated output into the workspace, which is on project storage. Scratch into /tmp,
↪  which vanishes with the job

We have 4 rtx 4090s, 64 CPUs, and 128GB of RAM available for use. We also prefetched the AFDB and the PDB. We
↪  don't have unlimited storage, so try to not exceed 500GB in retrieved/generated data.

/cluster/work/beltrao/shared on euler has:
alphafold/  - copy of the alphafold database
rsyncPDB/  - copy of the PDB
```

```
Literature: Design through repeated and independent critique: Search the current literature and available
↪  datasets, and write LITERATURE.md. Then draft DESIGN.md. The project goal and its parts are fixed, the
↪  suggestive ideas are a human priors, not a plan. Have the design reviewed and revised in at most three
↪  rounds by an external agentic PI, and agentic scientists assessing prior work, statistics, data,
↪  algorithms and compute, AI-agent failure modes, and other expertise the design needs. Each reviewer
↪  proposes concrete changes that stay within the project's scope, or states that none are needed.

Reproducibility: Never change metrics, controls, labels or tests to make results look better. Keep everything
↪  end-to-end reproducible: commit results to git, fix the random seeds list, and keep DATA.md listing the
↪  source, version, checksum and license of each dataset. Never commit restricted data, never hardcode data,
↪  prefer scripts over one-offs, make sources explicit, no silent assumptions. Don't trust a result until it
↪  has been checked. Write verification scripts using toy data with known answers, positive and negative
↪  controls, and sanity checks. An agent that didn't produce a result reviews it before you rely on it.

Research tree: Save this prompt verbatim to `PROMPT.md`. Keep track of the research in a research tree
↪  (RESEARCH_TREE.md), using a status legend: [ ] planned · [~] in progress · [x] done · [-]
↪  abandoned/negative · [!] key finding. At each milestone, add a short decision note and a two-paragraph
↪  status summary at the top. The tree is your memory: keep your own context lean by reading agents'
↪  summaries by default rather than raw logs and if your context is reset, resume from the tree.

AI lessons: Keep LESSONS.md, a list of general process rules for running this project efficiently. Whenever
↪  something wastes or saves significant effort, add a one-line rule with its evidence. At each milestone,
↪  prune it to the most useful rules and ask a fresh agent "what should we stop doing?" and "what worked
↪  well?".

Auto-research: Your job is to build the best possible method. Report runtime alongside accuracy. Keep going
↪  until you genuinely run out of promising ideas or close to the time limit. Then write REPORT.md with a
↪  breakdown of the approach, the best methods, test-set results, follow-ups, limitations, and remaining
↪  ideas.

Good luck!
```

9. Follow-up messages (verbatim)

The following is the content of manuscript/FOLLOWUP_PROMPTS.md.

```
The initial prompt is PROMPT.md (repository root). After the project's first completion the human sent these
↪  messages. Agent-to-agent briefs were not archived as files; they exist only in the local session
↪  transcripts, which are not distributed.

1. (after the first final summary)
   > I have some questions about the results.
   > 1. Does it work with multimers/dimers and was there any testing of this?
   > 2. Does it use GPU for running?
   > 3. Have we exhausted all ideas for speed optimzation and performance gains?

2. > Okay, we have some more time (3-4 hours) you can iterate some more.

3. > Are we done then? Is this finalized?

4. > You remain the PI of this computational project. Now your goal is to write the results of the project
↪  into a manuscript that explains the winning method and presents your benchmarking of it, formatted for
↪  the journal PLOS Computational Biology. This should be accompanied by an mvp github repository that will
↪  include the predictions and will allow readers to implement the method themselves (set up the repo and
↪  allow me to publish it myself). In both the paper and the repo, you should explicitly flag at the
↪  beginning that this predictive task was an activity in testing the ability of agents to run
↪  bioinformatics projects; the repo should include the original prompt, and the methods section should
↪  explain the agent and sub-agent setup. For the final method that worked, you should give credit to the
↪  ideas that you extracted from the literature. Write a supplementary file detailing the major methods you
↪  tried that did not work. In the paper, explain the method both conceptually and technically. Use this
↪  skill for writing https://github.com/jurgjn/euler-vibe/blob/main/home/euler-claude/.claude/skills/edit-fo
↪  r-clarity/SKILL.md. Mention and explain all important decisions you took and clearly explain your
↪  approach. Be critical and do not overstate. Stay precise, keep it short and to the point when possible.
↪  Make sure there is no personal or confidential data in the repo and write a sensible .gitignore.
```

```
Also: twice the session was restarted by the environment ("Continue" messages, no content), and the human did
↪  not otherwise intervene.
```